

Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра прикладної радіоелектроніки

Звіт
про лабораторну роботу №2 за темою
«Шифрування та розшифрування на машині «Енігма»»
з дисципліни
«Захист даних»

Викладач:

Навроцький Денис Олександрович

Виконав:

Студент групи РС-61мн

Гандзюк Дмитро Васильович

Київ – 2026 р.

Лабораторна робота №2

Шифрування та розшифрування на машині «Енігма»

Мета роботи: реалізувати шифр, побудований на двох базових криптографічних примітивах — перестановці (зсуві) та заміні (підстановці), — на прикладі спрощеної моделі шифрувальної машини «Енігма», довести коректність реалізації на офіційних валідаторах навчальної платформи CodinGame, а також самостійно оцінити криптографічну стійкість отриманого шифру: дослідити статистичні властивості шифротексту, визначити фактичний розмір простору ключів і практично реалізувати атаки в усіх основних моделях зломисника — від перебору ключа до відновлення відкритого тексту лише за шифротекстом.

Хід роботи

Постановка задачі

Задача взята з навчальної платформи CodinGame (розділ Training, рівень складності Easy) і має назву «Encryption/Decryption of Enigma Machine». У ній відтворено спрощений принцип роботи шифрувальної машини «Енігма», у якому обидва класичні криптографічні примітиви подані в чистому, нічим не замаскованому вигляді.

Перший примітив — **перестановка алфавіту**, реалізована як шифр Цезаря з інкрементним ключем: перший символ повідомлення зсувається на N позицій, другий — на $N+1$, третій — на $N+2$ і так далі. Другий примітив — **заміна**: отриманий після зсуву текст послідовно пропускається крізь три ротори, кожен з яких є довільною підстановкою (бієкцією) 26-літерного латинського алфавіту.

Формат вхідних даних задачі:

рядок 1	ENCODE DECODE
рядок 2	початковий зсув N , $0 \leq N < 26$
рядки 3–5	три ротори по 26 великих латинських літер
рядок 6	повідомлення, $1 \leq \text{довжина} < 50$, лише A..Z

Результатом роботи програми має бути одне слово — зашифроване або розшифроване повідомлення. Наведений в умові приклад зручно розібрати покроково: для повідомлення AAA та початкового зсуву $N = 4$ зсув дає EFG

(A+4, A+5, A+6), перший ротор перетворює це на JLC, другий — на BHD, третій — на KQF. Саме KQF і є очікуваною відповіддю.

Математична модель шифру

Позначимо літери числами від 0 до 25 ($A = 0$, $Z = 25$), а всі операції виконуватимемо в кільці лишків за модулем 26. Нехай $R1$, $R2$, $R3$ — підстановки, задані роторами, N — початковий зсув, m — відкритий текст, c — шифротекст. Тоді пряме та зворотне перетворення записуються так:

$$\begin{aligned}c[i] &= S[(m[i] + N + i) \bmod 26], & S &= R3 \cdot R2 \cdot R1 \\m[i] &= (S^{-1}[c[i]] - N - i) \bmod 26\end{aligned}$$

З цього запису впливають три спостереження, на яких надалі тримається вся дослідницька частина роботи.

Спостереження 1. Три ротори — це один ротор. Композиція бієкцій є знову бієкцією, тому добуток $S = R3 \cdot R2 \cdot R1$ — це одна підстановка алфавіту. Каскад із трьох роторів не дає нічого такого, чого не дав би один належно підібраний ротор. Це твердження перевірено не міркуванням, а програмно: метод `equivalent_single_rotor()` будує машину з одним ротором і нульовим зсувом, а модульний тест порівнює шифротексти обох машин на 50 випадкових конфігураціях — вони збігаються символ у символ.

Спостереження 2. Шифр є поліалфавітним із періодом рівно 26. На позиції i застосовується алфавіт заміни з номером $p = (N + i) \bmod 26$, причому всі 26 алфавітів породжені однією підстановкою S за правилом $A_p(m) = S[(m + p) \bmod 26]$. Отже, послідовність алфавітів циклічно повторюється з періодом 26 незалежно від вмісту повідомлення.

Спостереження 3. Увесь ключ стискається до 26 чисел. Введемо поняття *ефективного ключа* D , визначивши $D[x] = (S^{-1}[x] - N) \bmod 26$. Тоді розшифрування набуває вигляду $m[i] = (D[c[i]] - i) \bmod 26$. Це повний опис шифру: початковий зсув N «втоплено» всередину підстановки і окремо його знати не потрібно. Саме ефективний ключ D відновлюють усі чотири реалізовані далі атаки.

Програмна реалізація

Розв'язок виконано мовою Python 3 (основна реалізація) і продубльовано мовою C# на платформі .NET 8 (незалежна реалізація для

перехресної перевірки). Вихідний код самодостатнього розв’язку для платформи наведено в додатку А. Решта вихідних текстів — ядро шифру, модуль криптоаналізу, статистичні метрики, набір автоматичних тестів і реалізація мовою C# — доступні в репозиторії роботи, посилання на який наведено в списку використаних джерел.

Головне архітектурне рішення впливає зі спостереження 1: каскад роторів згортається в одну таблицю підстановки до початку обробки повідомлення. Завдяки цьому у внутрішньому циклі на кожен символ припадає лише одне звертання до таблиці. У повній версії бібліотеки попередньо обчислюються всі 26 алфавітів заміни (матриця 26×26 , тобто 676 байт), тому внутрішній цикл узагалі не містить арифметичних операцій — тільки індексацію.

Обчислювальна складність шифрування становить $O(L + 26k)$, де L — довжина повідомлення, а k — кількість роторів; додаткова пам’ять — $O(26^2)$ і не залежить від довжини повідомлення. Розшифрування симетричне та має ту саму складність. Ядро шифру й усі атаки написані виключно на стандартній бібліотеці Python, без зовнішніх залежностей.

Верифікація розв’язку

Правильність реалізації підтверджено на чотирьох незалежних рівнях: офіційними валідаторами платформи, збереженими в репозиторії офіційними тестами, набором властивісних тестів і перехресним порівнянням двох реалізацій різними мовами.

Розв’язок надіслано на перевірку платформі CodinGame. Усі шість видимих тестів пройдено, і, що важливіше, пройдено приховані валідатори: підсумкова оцінка становить **100 %**. Надісланий код у вікні редактора платформи наведено на рисунку 2.1, результат прогону всіх шести тестів — на рисунку 2.2, підсумкову оцінку валідаторів — на рисунку 2.3.

```

1
2 import sys
3
4 A = ord("A")
5 M = 26
6
7
8 def main() -> None:
9     data = sys.stdin.read().split()
10    mode = data[0].upper()
11    shift = int(data[1]) % M
12    rotors = data[2:5]
13    message = data[5] if len(data) > 5 else ""
14
15    # Collapse the rotor cascade into one substitution table.
16    s = list(range(M))
17    for wiring in rotors:
18        table = [ord(ch) - A for ch in wiring]
19        s = [table[x] for x in s]
20
21    if mode == "ENCODE":
22        out = [
23            chr(A + s[(ord(ch) - A + shift + i) % M])
24            for i, ch in enumerate(message)
25        ]
26    else:
27        inv = [0] * M
28        for i, v in enumerate(s):
29            inv[v] = i
30        out = [
31            chr(A + (inv[ord(ch) - A] - shift - i) % M)
32            for i, ch in enumerate(message)
33        ]
34
35    print("".join(out))
36
37
38 if __name__ == "__main__":
39     main()
40

```

Рисунок 2.1 – Надісланий розв’язок у вікні редактора CodinGame

Test cases			
01	Encode 3		▶ PLAY TESTCASE
02	Encode 23		▶ PLAY TESTCASE
03	Decode 21		▶ PLAY TESTCASE
04	Encode 21		▶ PLAY TESTCASE
05	Encode 42		▶ PLAY TESTCASE
06	Decode 49		▶ PLAY TESTCASE

Рисунок 2.2 – Усі шість тестів платформи позначені як пройдені

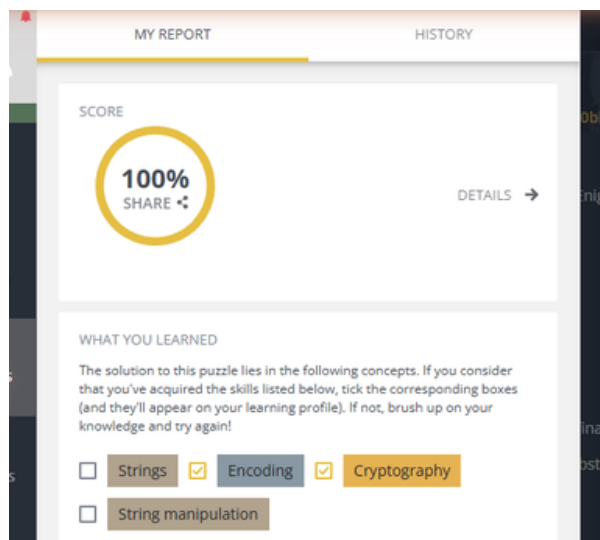


Рисунок 2.3 – Звіт платформи: оцінка валідаторів — 100 %

Входи та еталонні виходи всіх шести офіційних тестів збережено в репозиторії у файлі `tests/official_cases.json`, тому їх можна прогнати локально без доступу до платформи. Перелік тестів наведено в таблиці 2.1.

Таблиця 2.1 – Офіційні тести CodinGame

№	Назва	Режим	N	Довжина	Результат
1	Encode 3	ENCODE	4	3	пройдено
2	Encode 23	ENCODE	7	23	пройдено
3	Decode 21	DECODE	9	21	пройдено
4	Encode 21	ENCODE	9	21	пройдено
5	Encode 42	ENCODE	9	42	пройдено
6	Decode 49	DECODE	5	49	пройдено

Результат локального прогону цих тестів:

```
$ python run.py selftest
Офіційні тести CodinGame
```

```
-----
[OK] Encode 3      ENCODE -> KQF
[OK] Encode 23     ENCODE -> ALWAURKQEQQWLRAWZHUYKVN
[OK] Decode 21     DECODE -> EVERYONEISWELCOMHERE
[OK] Encode 21     ENCODE -> PQSACVVTOISXFXCIAMQEM
[OK] Encode 42     ENCODE -> PQSACVVTOISXFXCIAMQEMDZIXFJJSTQIENEFQXVZYV
[OK] Decode 49     DECODE -> THEQUICKBROWNFOXJUMPSOVERALAZYSphinxOFBLACKQUARTZ
-----
```

Пройдено 6 з 6

Окрім офіційних тестів, написано набір із 86 автоматичних тестів, що перевіряють: оборотність перетворення на 300 випадкових конфігураціях; періодичність шифру (перші 26 символів шифротексту рядка з 52 літер А збігаються з наступними 26); еквівалентність каскаду з трьох роторів одному ротору; модульність зсуву ($N = 3$ і $N = 29$ дають однаковий результат); некомутативність порядку роторів; відхилення некоректних роторів і символів поза алфавітом А..Z. Реалізація мовою С# проходить ті самі шість офіційних тестів і 2000 перевірок оборотності, а окремий тест порівнює вихід обох реалізацій символ у символ. Увесь набір виконується автоматично при кожній зміні коду засобами GitHub Actions на трьох версіях Python.

Під час підготовки роботи в самому тестовому наборі було виявлено й усунуто помилку: вивід програми на С# читався в кодуванні консолі Windows замість UTF-8, через що порівняння з українським рядком не спрацьовувало. Це показове нагадування, що тест, який ніколи не запускався, не є доказом

нічого.

Частотний аналіз шифротексту

Перше, що перевіряє криптоаналітик, — чи зберігає шифротекст частотну структуру мови. Для експерименту взято англomовний текст обсягом 2035 літер і зашифровано його з початковим зсувом $N = 7$. Порівняння частот літер відкритого тексту, шифротексту й еталонних частот англійської мови наведено на рисунку 2.4.

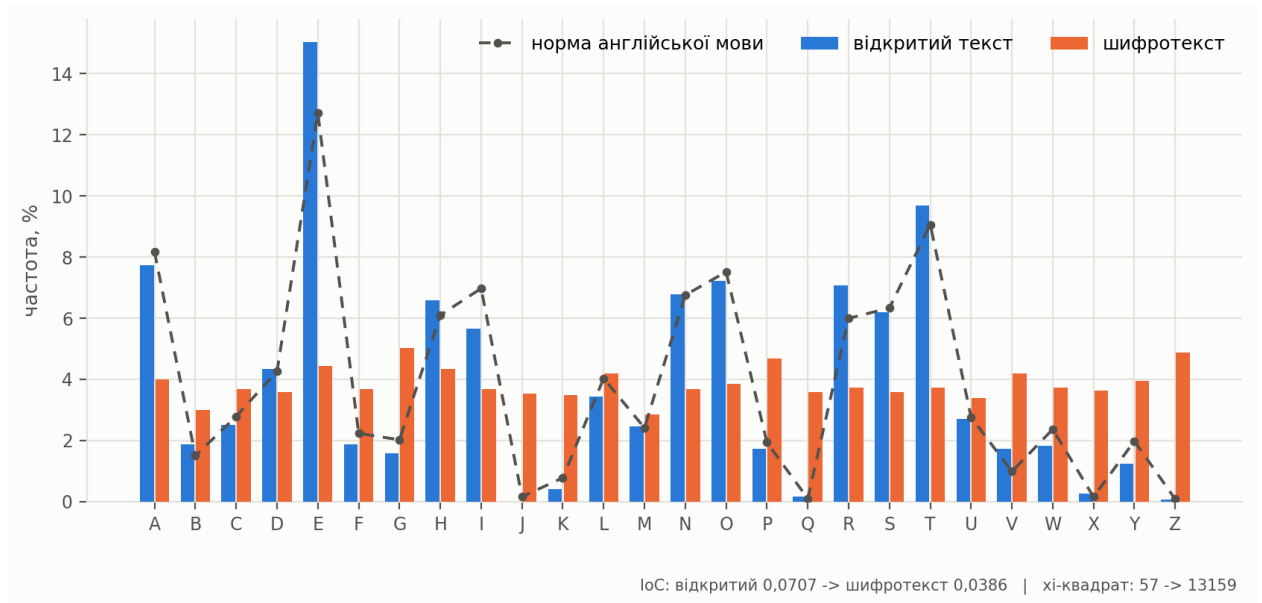


Рисунок 2.4 – Інкрементний зсув вирівнює частоти літер шифротексту

Результат на перший погляд свідчить на користь шифру. Інкрементний зсув «розмазує» кожен літеру відкритого тексту по всіх 26 позиціях алфавіту, і розподіл літер шифротексту стає майже рівномірним: максимальне відхилення частоти будь-якої літери від рівномірних 3,85 % становить лише 1,17 відсоткового пункту. Кількісні показники зведено в таблиці 2.2.

Таблиця 2.2 – Статистичні характеристики тексту до та після шифрування

Показник	Відкритий текст	Шифротекст	Орієнтир
Індекс відповідності (IoC)	0,0707	0,0386	англ. 0,0667 / випадк. 0,0385
Критерій χ^2 -квадрат	57	13159	менше — ближче до мови
Ентропія, біт/символ	4,105	4,689	максимум 4,700

Індекс відповідності впав з 0,0707 (типове значення для англійської мови) до 0,0386, що практично збігається зі значенням для рівномірно випадкової послідовності. Ентропія зросла до 4,689 біта на символ при теоретичному максимумі 4,700. Класичний частотний аналіз тут безсилий: шифротекст статистично невідрізнити від шуму.

Саме на цьому місці й виникає найпоширеніша помилка — висновок, що шифр стійкий. Наступні розділи показують, що це не так.

Визначення періоду шифру

Ключ до злому дає індекс відповідності, застосований не до всього тексту, а до його частин. Розіб'ємо шифротекст на p стовпців за індексом $i \bmod p$. Якщо припущений період p збігається зі справжнім, то кожен стовець виявиться результатом однієї й тієї самої моноалфавітної заміни, а індекс відповідності всередині стовпця повернеться до значення, характерного для природної мови (ІоС інваріантний щодо простої заміни). Профіль залежності середнього ІоС стовпця від припущеного періоду наведено на рисунку 2.5.

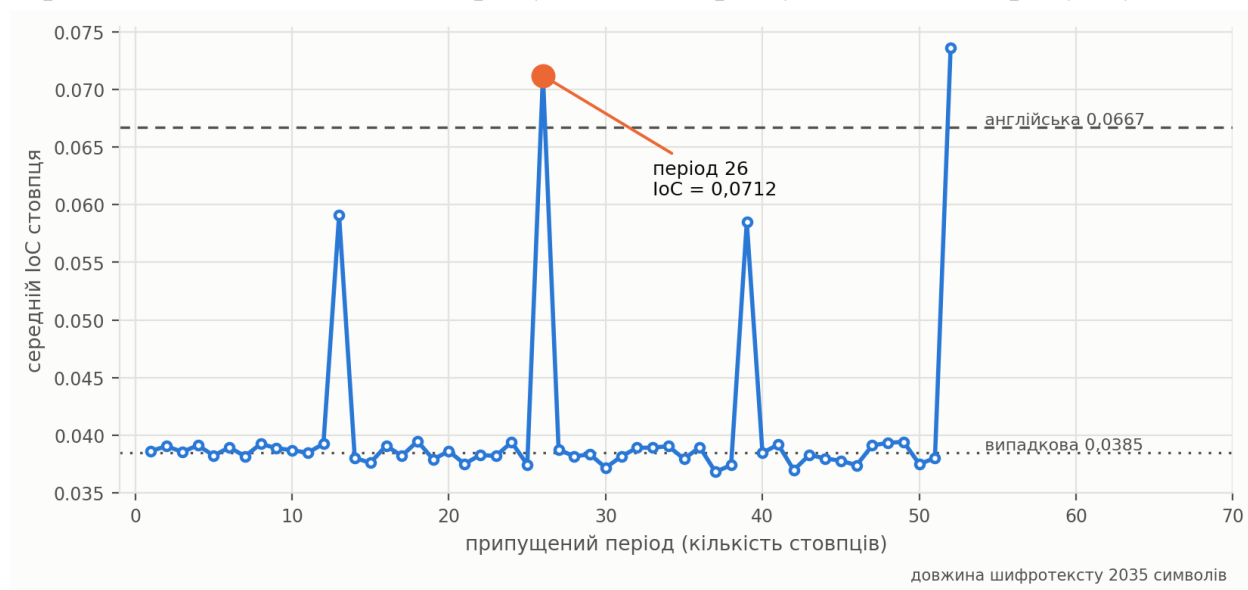


Рисунок 2.5 – Профіль індексу відповідності викриває поліалфавітний період 26

На профілі чітко видно сплеск рівно на періоді 26 (ІоС = 0,0712 проти 0,0386 для тексту в цілому) та на кратних йому значеннях 13, 39 і 52. Це експериментальне підтвердження спостереження 2, отримане без будь-якого знання про ротори чи початковий зсув. Отже, перший крок криптоаналізу — визначення структури шифру — виконується автоматично.

Ієрархія атак на шифр

Реалізовано чотири атаки, впорядковані за спаданням припущень про можливості зловмисника — від найсильніших до найслабших. Усі вони відновлюють один і той самий об'єкт — ефективний ключ D. Для оцінювання правдоподібності кандидатів використано статистичну модель англійської мови на квадриграмах, натреновану на корпусі обсягом 5 623 458 літер текстів суспільного надбання (у репозиторії зберігається лише похідна статистика частот, а не самі тексти). Зведені результати наведено в таблиці 2.3.

Таблиця 2.3 – Результати атак на повідомленні завдовжки 180 символів

Атака	Що відомо зловмиснику	Вартість	Час	Ключ
Перебір зсуву	ротори	26 варіантів	0,007 с	відновлено
Відомий відкритий текст	пара текстів	180 символів	< 0,001 с	відновлено
Підібраний відкритий текст	доступ до оракула	1 запит (26 символів)	< 0,001 с	відновлено
Тільки шифротекст	нічого, крім шифротексту	78 481 оцінка	3,04 с	відновлено

Атака 1 — повний перебір зсуву

Найважливіше зауваження до цієї задачі впливає з принципу Керкгоффа: стійкість шифру має залежати виключно від ключа, а не від секретності алгоритму. У розглянутій задачі ротори надруковані просто в умові, і, що показово, в усіх шести офіційних тестах вони однакові (це зафіксовано окремим тестом). Отже, справжнім секретом лишається єдине число — початковий зсув N.

Це дає лише 26 варіантів ключа, тобто близько 4,7 біта ентропії. Атака полягає в розшифруванні шифротексту всіма 26 ключами та виборі найбільш «англійського» результату. Розподіл оцінок правдоподібності по кандидатах наведено на рисунку 2.6.

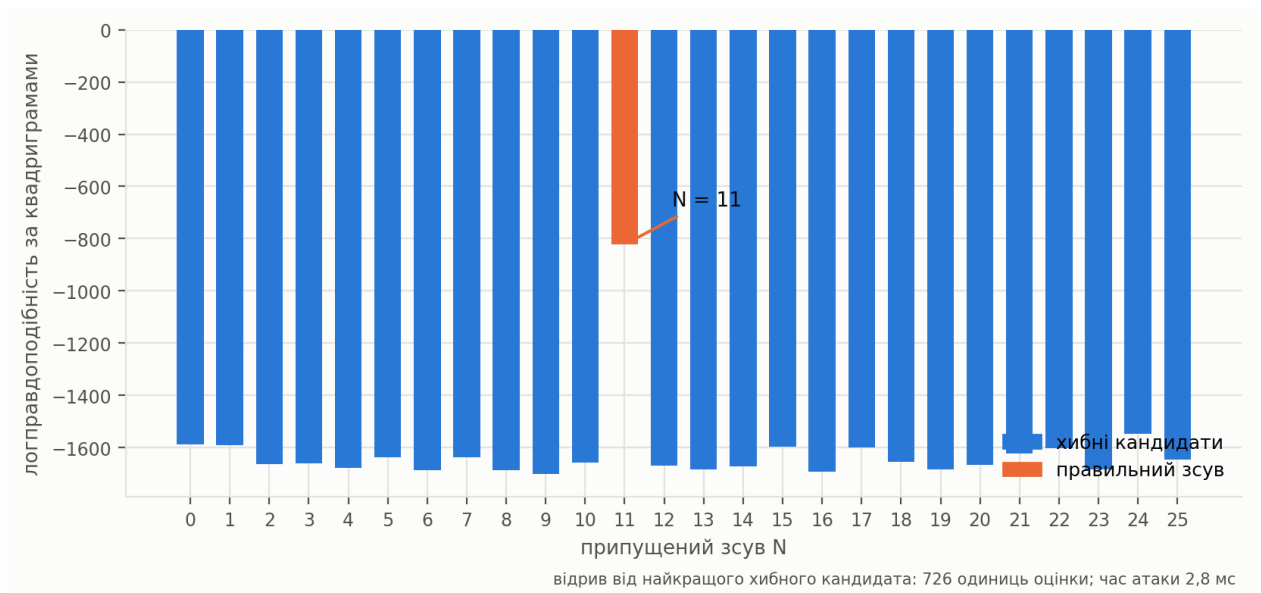


Рисунок 2.6 – За відомих роторів правильний ключ визначається однозначно з першої спроби

Перебір завершується за 2,8 мілісекунди, а правильний кандидат відривається від найкращого хибного на 726 одиниць логарифмічної правдоподібності, тож жодної неоднозначності не виникає. Атака успішна на 100 % повідомлень будь-якої довжини, включно з трисимвольними, — тобто в умовах цієї задачі шифр зламується повністю й миттєво.

Атака 2 — відомий відкритий текст

Якщо зломиснику відомий хоча б фрагмент відкритого тексту разом з відповідним фрагментом шифротексту (так звана «шпаргалка», *crib*), ключ відновлюється взагалі без перебору: кожна відома пара символів дає один елемент ефективного ключа за формулою $D[c[i]] = (m[i] + i) \bmod 26$.

Виникає природне запитання: скільки відомого тексту для цього потрібно? Задача зводиться до класичної задачі про збирання купонів, для якої очікуване покриття описується формулою $26 \cdot (1 - (25/26)^L)$. Експериментальна перевірка (по 200 прогонів на кожну точку) показала збіг із теорією із середнім відхиленням 0,061 елемента ключа — рисунок 2.7 і таблиця 2.4.

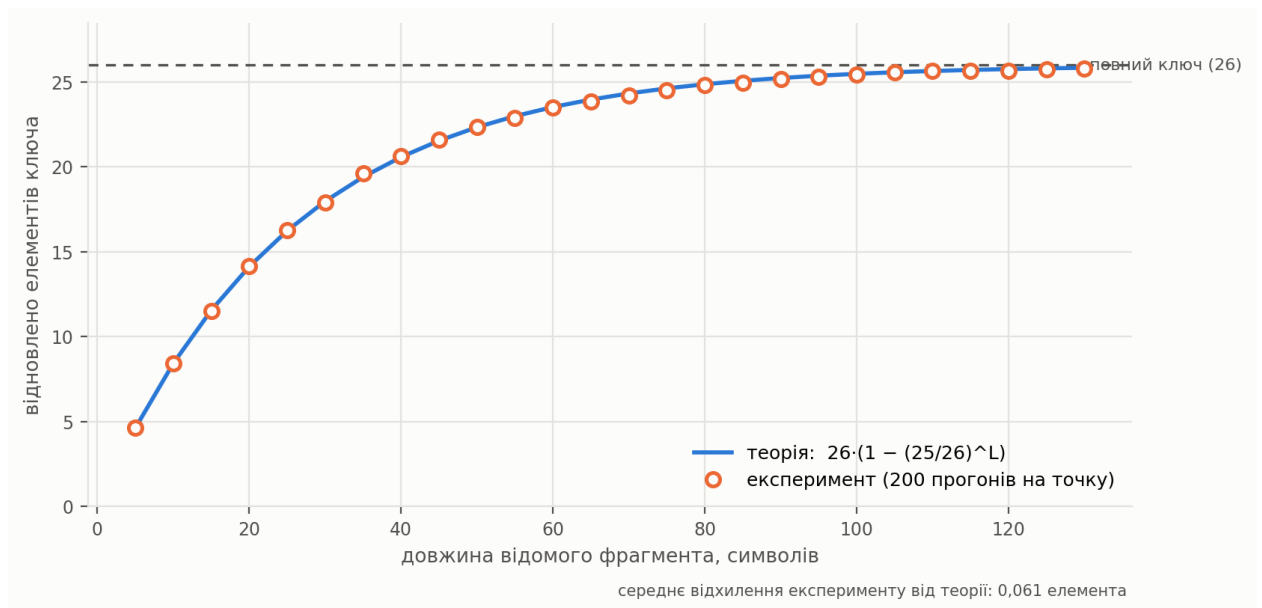


Рисунок 2.7 – Відновлення ключа за відомим відкритим текстом:
експеримент проти теорії

Таблиця 2.4 – Покриття ефективного ключа залежно від довжини відомого фрагмента

Довжина фрагмента L	10	25	50	105
Експеримент	8,44	16,29	22,35	25,56
Теорія	8,44	16,25	22,34	25,58

Щоб зібрати всі 26 елементів ключа, потрібно в середньому близько $26 \cdot H(26) \approx 100$ символів відомого тексту. Проте повний ключ зазвичай і не потрібен: навіть половина елементів дозволяє прочитати більшу частину повідомлення, а решту символів легко відновити за контекстом.

Атака 3 — підібраний відкритий текст

Найдешевша з реалізованих атак. Припустимо, що злоумисник має доступ до шифрувального пристрою як до «чорної скриньки» (оракула) і може подати на вхід довільне повідомлення. Надішлемо рядок із 26 однакових літер А. Тоді $c[i] = S[(0 + N + i) \bmod 26]$, а отже $D[c[i]] = i$ для всіх i від 0 до 25.

Оскільки S — бієкція, усі 26 отриманих символів різні, і повний ефективний ключ відновлюється з єдиного запиту завдовжки **26 символів**. Ані ротори, ані початковий зсув при цьому знати не потрібно. Коректність атаки перевірено на 25 випадкових конфігураціях машини — у кожному

випадку відновлений ключ точно збігся зі справжнім.

Атака 4 — відновлення тексту лише за шифротекстом

Це найслабша модель зломисника і, відповідно, найцікавіший результат роботи: відомий лише шифротекст, ротори й зсув невідомі повністю. Ключова ідея полягає в тому, що позиційний зсув *-і є публічною структурою, а не секретом*: його не треба вгадувати. Тому перебирати доводиться лише одну бієкцію D , і задача зводиться до звичайної моноалфавітної заміни.

Реалізовано сходження за схилом (hill climbing) у просторі бієкцій з фітнес-функцією на квадриграмах. Початкове наближення будується жадібним частотним методом: для кожної пари «літера шифротексту — кандидат» обчислюється частотна правдоподібність породжених символів, після чого пари розбираються за спаданням вигоди зі збереженням бієктивності. Далі ключ уточнюється випадковими обмінами з кількома незалежними перезапусками. Залежність частки повних зламів від довжини повідомлення наведено на рисунку 2.8 і в таблиці 2.5.

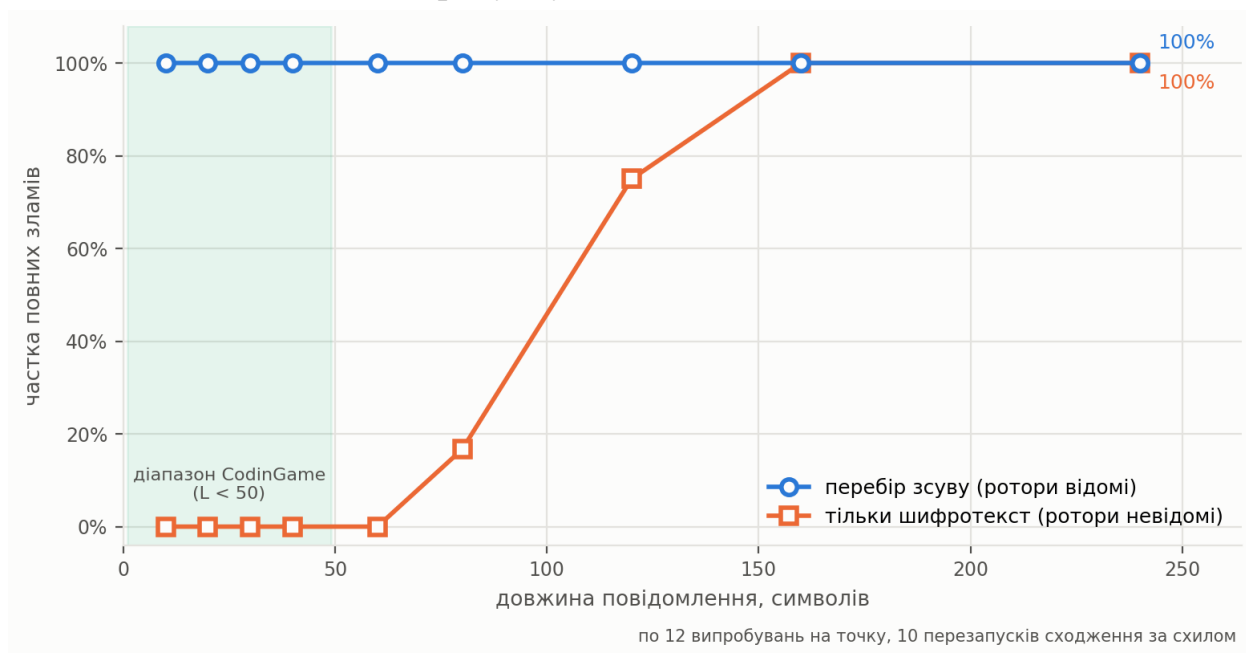


Рисунок 2.8 – Успішність атак залежно від довжини повідомлення

Таблиця 2.5 – Успішність атаки лише за шифротекстом

Довжина повідомлення, символів	40	80	120	160	240
Частка повних зламів, %	0	17	75	100	100

Довжина повідомлення, символів	40	80	120	160	240
Середній час атаки, с	0,24	0,49	0,74	1,20	1,06

Поріг має природне пояснення. Відстань єдиності за Шенноном для ключа обсягом $26!$ обчислюється як $U = H(K)/D = 88,4 / (4,70 - 1,5) \approx 28$ символів. Це означає, що шифротексти, коротші за приблизно 28 символів, у принципі мають кілька осмислених розшифрувань і не можуть бути однозначно зламані жодним алгоритмом. Отриманий практичний поріг (близько 120–160 символів) у 4–6 разів вищий за теоретичну межу, що є типовим розривом між інформаційно-теоретичною оцінкою та можливостями конкретного евристичного алгоритму.

Окремо варто зазначити наслідок для умови задачі: платформа обмежує довжину повідомлення 50 символами, тобто саме в тому діапазоні, де ціліснотекстова атака ще не працює надійно. Проте це не рятує шифр, бо ротори в задачі є публічними, і атака 1 ламає 100 % таких повідомлень за мілісекунди.

Оцінка простору ключів

Номінально ключем шифру є три довільні підстановки 26-літерного алфавіту та початковий зсув, тобто $(26!)^3 \cdot 26$ варіантів, що відповідає $2^{269,8}$. Проте, як показано у спостереженнях 1 і 3, весь цей набір згортається в одну бієкцію D , і фактичний простір ключів становить лише $26!$, тобто $2^{88,4}$. Якщо ж урахувати, що ротори опубліковані в умові, залишається 26 варіантів — $2^{4,7}$. Порівняння наведено на рисунку 2.9 і в таблиці 2.6.

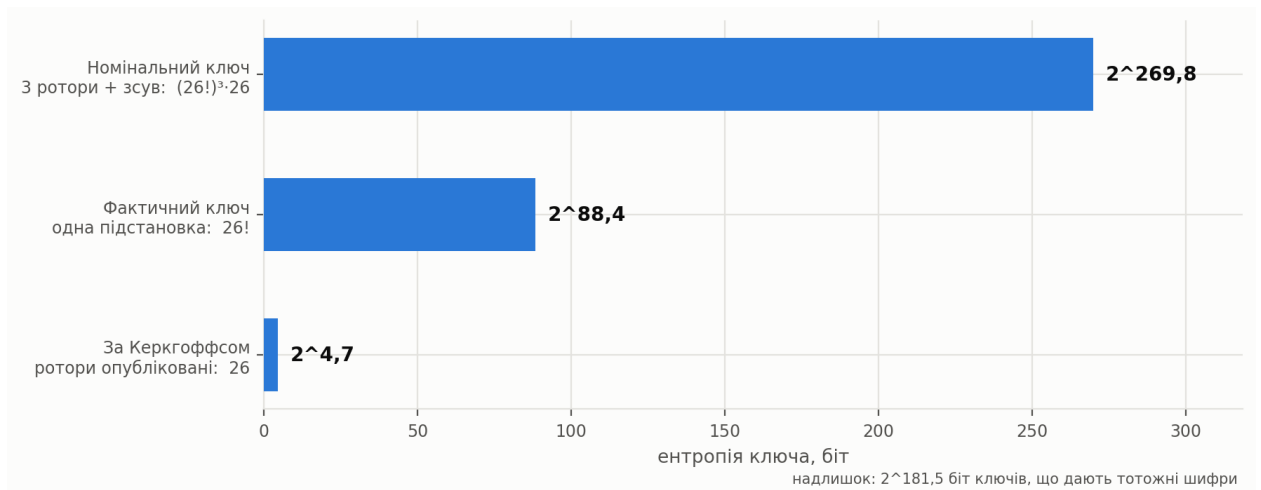


Рисунок 2.9 – Заявлений і фактичний розмір простору ключів

Таблиця 2.6 – Розмір простору ключів у різних моделях

Модель	Простір ключів	Ентропія ключа
Номинально: 3 ротори та зсув	$(26!)^3 \cdot 26$	$2^{269,8}$
Фактично: одна підстановка D	$26!$	$2^{88,4}$
За Керкгоффсом: ротори відомі	26	$2^{4,7}$

Отже, з заявлених $2^{269,8}$ біта корисними є лише $2^{88,4}$: рівно $2^{181,5}$ різних наборів «ротори + зсув» дають тотожні шифротексти. Це твердження доведено не лише формулою, а й конструктивно: функція `find_key_collision()` будує два різні ключі, шифротексти яких збігаються символ у символ на 120 випадкових літерах.

Дослідження дифузії

Для стійкого шифру виконується строгий лавинний критерій: зміна одного біта відкритого тексту має змінювати близько половини шифротексту. Виміряємо цю величину для досліджуваного шифру, змінюючи один символ відкритого тексту та підраховуючи кількість змінених символів шифротексту (по 120 випробувань на кожну точку). Результати наведено на рисунку 2.10 і в таблиці 2.7.

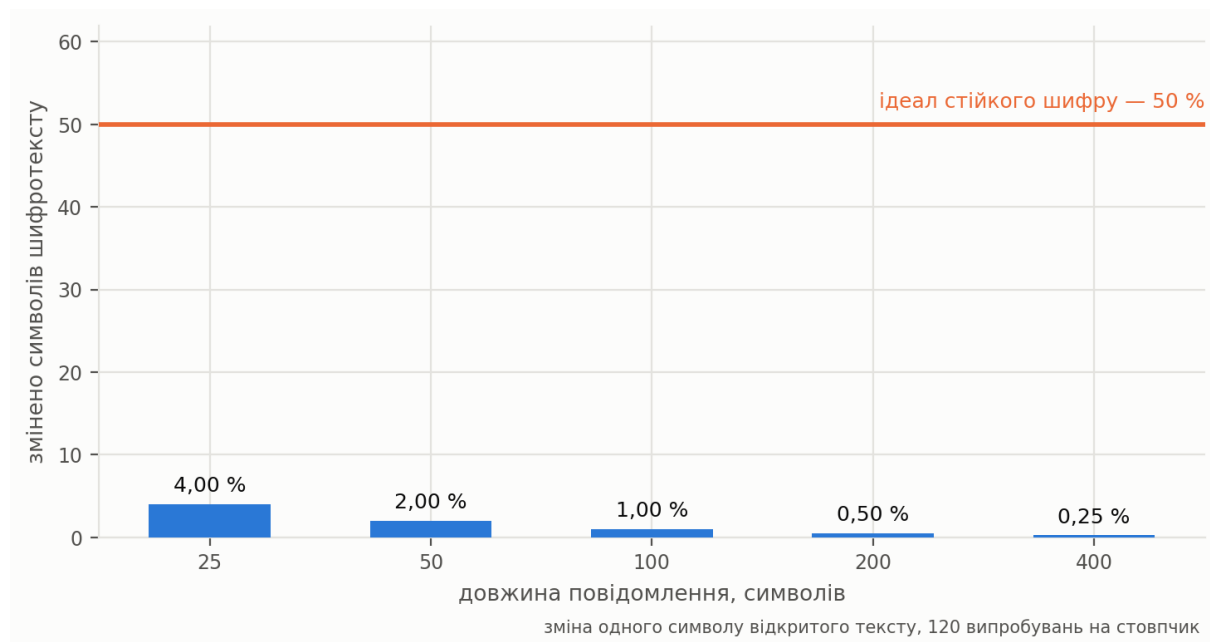


Рисунок 2.10 – Дифузія відсутня: один символ впливає рівно на один

Таблиця 2.7 – Лавинний ефект

Довжина повідомлення, символів	25	50	100	200	400
Змінено символів шифротексту	1	1	1	1	1
Частка зміненого тексту, %	4,00	2,00	1,00	0,50	0,25

Результат однозначний: змінюється **рівно один** символ, і так для повідомлень будь-якої довжини. Для тексту з 400 символів це 0,25 % замість очікуваних 50 %. Дифузія відсутня повністю — кожен символ шифрується незалежно від решти. Практичні наслідки серйозні: зловмисник може редагувати шифротекст посимвольно, а часткове знання ключа одразу дає частковий відкритий текст, без «розсіпання» решти повідомлення.

Порівняння зі справжньою машиною «Енігма»

Під час роботи виявлено цікавий факт: набір роторів з умови задачі — це справжні проводки роторів машини Enigma I (Wehrmacht, 1930 рік), але перелічені у зворотному порядку. Ротору, названому в умові «**ROTOR I**», відповідає історичний ротор III, «**ROTOR II**» — історичний ротор II, а «**ROTOR III**» — історичний ротор I.

Попри це, розглянута модель не є машиною «Енігма», а лише її суттєвим спрощенням. Основні відмінності зведено в таблиці 2.8.

Таблиця 2.8 – Порівняння справжньої Enigma I та моделі із задачі

Властивість	Справжня Enigma I	Модель із задачі
Обертання роторів	так, після кожного символу	ні, ротори нерухомі
Період шифру	$26 \cdot 25 \cdot 26 = 16\,900$	26
Рефлектор	так (UKW-B/C)	немає
Комутаційна панель	так, 10 пар кабелів	немає
Вибір і порядок роторів	3 з 5, тобто 60 варіантів	задається на вході
Початкові положення	$26^3 = 17\,576$	один зсув, 26
Самошифрування літери	неможливе (вада рефлектора)	можливе

Виникає парадокс, який варто підкреслити окремо: за «сирим» розміром простору ключів спрощена модель виглядає навіть сильнішою за історичну машину. Проте історичну «Енігму» зламували саме через її структурні вади — наявність рефлектора й неможливість переходу літери саму в себе, — а розглянуту модель ламає відсутність руху роторів, яка

перетворює її на моноалфавітну заміну з передбачуваною надбудовою.
Розмір ключа сам собою не гарантує нічого.

Висновки

1. Реалізовано шифр, що поєднує два базові криптографічні примітиви — перестановку (інкрементний зсув Цезаря) і заміну (каскад із трьох роторів). Розв’язок надіслано на платформу CodinGame й отримано оцінку валідаторів 100 %: пройдено всі шість офіційних тестів, включно з прихованими. Реалізацію продубльовано двома мовами (Python і C#) та покрито 86 автоматичними тестами, які виконуються в системі безперервної інтеграції.

2. Алгоритм оптимізовано не мікрооптимізаціями, а структурно: розуміння того, що композиція роторів є однією підстановкою, дає лінійну складність $O(L)$ без арифметичних операцій у внутрішньому циклі та сталий додатковий пам’ять.

3. Головний теоретичний результат роботи: два примітиви, застосовані послідовно й незалежно, не додають стійкості один одному. Заміна дає 26! варіантів, зсув — 26, але оскільки зсув детермінований і публічний, сумарна стійкість дорівнює стійкості однієї заміни.

4. Кількісно: заявлений простір ключів $2^{269,8}$ фактично дорівнює $2^{88,4}$, а в умовах задачі (опубліковані ротори) — лише $2^{4,7}$, тобто менш ніж 5 біт. Надлишковість у $2^{181,5}$ ключів доведено конструктивно побудованою колізією.

5. Шифр зламується в усіх чотирьох моделях атак. Найдешевша атака (підібраний відкритий текст) відновлює повний ключ за один запит із 26 символів. Найслабша за припущеннями атака — лише за шифротекстом, без будь-якого знання ключа — відновлює текст за приблизно секунду для повідомлень від 160 символів.

6. Основна вада конструкції — нерухомі ротори. Саме обертання роторів робило історичну «Енігму» поліалфавітним шифром із періодом 16 900; без нього лишається моноалфавітна заміна, замаскована передбачуваним позиційним зсувом.

7. Практичний урок роботи: висока ентропія шифротексту (4,689 з 4,700 біта на символ) і «випадковий» вигляд частотного розподілу не є доказом стійкості. Стійкість визначає структура ключа, а не зовнішній вигляд виходу шифратора.

Список використаних джерел

1. CodinGame. Encryption/Decryption of Enigma Machine [Електронний ресурс]. – Режим доступу: <https://www.codingame.com/training/easy/encryption-decryption-of-enigma-machine>
2. Caesar cipher [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Caesar_cipher
3. Enigma machine [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_machine
4. Enigma rotor details [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_rotor_details
5. Index of coincidence [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Index_of_coincidence
6. Shannon C. E. Communication Theory of Secrecy Systems // Bell System Technical Journal. – 1949. – Vol. 28, No. 4. – P. 656–715.
7. Beker H., Piper F. Cipher Systems: The Protection of Communications. – London: Northwood Books, 1982. – 436 p.
8. Singh S. The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography. – New York: Anchor Books, 2000. – 411 p.
9. Вихідний код роботи [Електронний ресурс]. – Режим доступу: <https://github.com/RE22GDV/LR2-Enigma>

Додаток А

Розв'язок задачі, надісланий на платформу CodinGame

Нижче наведено повний текст файла `solution/codingame_solution.py` — саме цей код надіслано на платформу CodinGame й оцінено валідаторами на 100 %. Решта вихідних текстів роботи (ядро шифру, модуль криптоаналізу, статистичні метрики, набір із 86 автоматичних тестів, скрипти експериментів і незалежна реалізація мовою C#) не наводиться тут, щоб не переобтяжувати звіт, і доступна в репозиторії: <https://github.com/RE22GDV/LR2-Enigma>

```
"""
CodinGame - "Encryption/Decryption of Enigma Machine" (Easy)
https://www.codingame.com/training/easy/encryptiondecryption-of-enigma-machine

Self-contained solution, submitted as-is (validator score: 100%).
This file is kept ASCII-only and byte-identical to the accepted submission;
the Ukrainian write-up lives in README.md and in src/enigma/.

Input format:
  line 1   : ENCODE | DECODE
  line 2   : starting shift N
  lines 3-5: three rotors (26 uppercase letters each)
  line 6   : the message

Encryption:
  c[i] = S[ (m[i] + N + i) mod 26 ],    S = R3 . R2 . R1
Decryption (exact inverse):
  m[i] = ( S^-1[c[i]] - N - i ) mod 26

The three rotors are composed into a single substitution S *before* the
message loop, because a composition of bijections is again a bijection.
That leaves exactly one table lookup per character:
time O(L + 26k), extra memory O(26).
"""

import sys

A = ord("A")
M = 26

def main() -> None:
    data = sys.stdin.read().split()
    mode = data[0].upper()
    shift = int(data[1]) % M
    rotors = data[2:5]
    message = data[5] if len(data) > 5 else ""

    # Collapse the rotor cascade into one substitution table.
    s = list(range(M))
    for wiring in rotors:
        table = [ord(ch) - A for ch in wiring]
        s = [table[x] for x in s]

    if mode == "ENCODE":
        out = [
```

```

        chr(A + s[(ord(ch) - A + shift + i) % M])
        for i, ch in enumerate(message)
    ]
else:
    inv = [0] * M
    for i, v in enumerate(s):
        inv[v] = i
    out = [
        chr(A + (inv[ord(ch) - A] - shift - i) % M)
        for i, ch in enumerate(message)
    ]

print("".join(out))

if __name__ == "__main__":
    main()

```